

Gesture Control Car Action Group (No-PTZ)

Experiment Objective

Recognize 5 gestures to trigger preset chassis action sequences (square/circle/S-curve/back-and-forth/stop), with odometry-driven PD closed-loop control for completion determination. The No-PTZ version uses a fixed camera bracket.

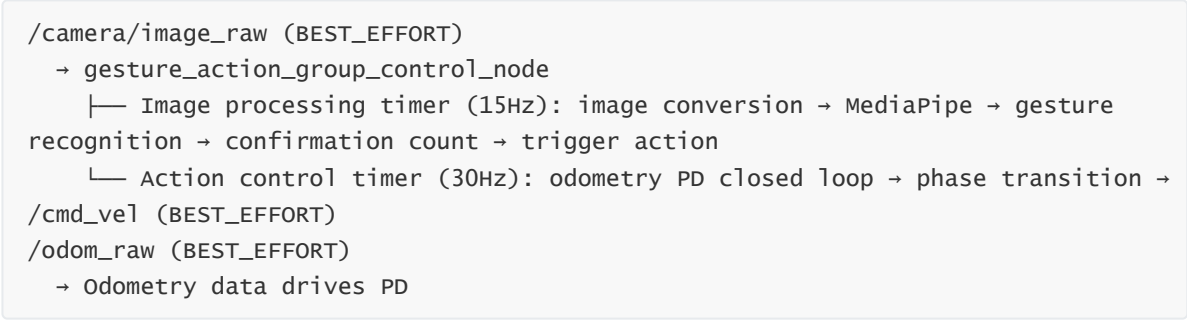
Experiment Principle

1. Keyword Explanation

Keyword	Description
Action Group	Preset chassis motion sequences, with odometry data driving phase transitions
Odometry Closed-Loop PD	Subscribes to <code>/odom_raw</code> ; each phase uses PD to drive toward the target distance/angle, then cools down before switching phases after settling
Gesture Confirmation Count	A gesture must be held for <code>gesture_hold_confirm_frames</code> (3) consecutive frames to trigger, preventing false triggering
Phase Cooldown	After PD settles, the car stops for <code>phase_cooldown_ticks</code> (10) ticks to let the robot stabilize before switching phases
Three-State Machine	idle → waiting (waiting for gesture) → action_running (executing action); Space toggles idle↔waiting
5-Finger Emergency Stop	In any state, extending all five fingers interrupts the current action and returns to waiting
QoS	best_effort (image/odometry input + cmd_vel downlink), depth=1
TF Coordinate Frames	odom → base_footprint → base_link

2. Technical Architecture

Three-state machine + dual timers (image 15Hz + action control 30Hz):



Gesture → action mapping:

Gesture	Finger code	Action
Yes	[0,1,1,0,0]	square (0.5m × 4 sides)
OK	≥3 fingers with index bent	circle (rotate 360°)
Rock	[0,1,0,0,1]	s_curve (S-curve)
Thumb Down	≤1 finger with thumb down	backforth (back-and-forth 0.5m)
5 fingers	[1,1,1,1,1]	stop (emergency stop)

PD closed-loop parameters:

- Distance phase: $\text{linear_kp}(0.8) \times \text{error} + \text{linear_kd}(0.25) \times \text{derivative}$ → clamped to ± 0.2 m/s
- Angle phase: absolute yaw target, $\text{angular_kp}(0.05) \times \text{error} + \text{angular_kd}(0.005) \times \text{derivative}$ → clamped to ± 0.6 rad/s
- Settle for 3 frames → cooldown for 10 ticks → switch phase

3. Core Topics Quick Reference

Firmware Uplink (Sensor Data)

Topic	Message Type	QoS	Description
/camera/image_raw	sensor_msgs/Image	best_effort	ESP32 camera raw image frames
/odom_raw	nav_msgs/Odometry	best_effort	Raw odometry data (drives the PD closed loop)

Firmware Downlink (Control Commands)

Topic	Message Type	QoS	Field	Range	Direction
/cmd_vel	geometry_msgs/Twist	best_effort + keep_last(1)	linear.x	-0.20~0.20 m/s	$\pm$ forward, $\mp$ backward
			angular.z	-0.60~0.60 rad/s	$\pm$ turn left, $\mp$ turn right

Experiment Steps

1. Hardware Preparation

Hardware	Requirement
PTZ version	✗ This section is for the No-PTZ version
No-PTZ version	☑
Required peripherals	ESP32 camera (fixed bracket)

2. Start Agent + Camera + Joint Model (Terminal 1)

Start agent + camera + joint model (Terminal 1)

```
ros2 launch microros_v2_bringup car_base.launch.py
```

```
yahboom@yahboom:~$ ros2 launch microros_v2_bringup car_base.launch.py
[INFO] [launch]: All log files can be found below /home/yahboom/.ros/log/2026-07-28-15-04-46-269642-yahboom-579370
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [micro_ros_agent-1]: process started with pid [579411]
[INFO] [camera_driver-2]: process started with pid [579413]
[INFO] [robot_state_publisher-3]: process started with pid [579415]
[INFO] [joint_state_publisher-4]: process started with pid [579417]
[INFO] [ekf_node-5]: process started with pid [579419]
[micro_ros_agent-1] [1785222287.404236] info      | UDPv4AgentLinux.cpp | init          | running...      | port: 8090
[robot_state_publisher-3] [INFO] [1785222288.847816851] [robot_state_publisher]: got segment base_footprint
[robot_state_publisher-3] [INFO] [1785222288.848013042] [robot_state_publisher]: got segment base_link
[robot_state_publisher-3] [INFO] [1785222288.848028942] [robot_state_publisher]: got segment bwheel_Link
[robot_state_publisher-3] [INFO] [1785222288.848037122] [robot_state_publisher]: got segment can1_Link
[robot_state_publisher-3] [INFO] [1785222288.848043874] [robot_state_publisher]: got segment can2_Link
[robot_state_publisher-3] [INFO] [1785222288.848050538] [robot_state_publisher]: got segment can3_Link
[robot_state_publisher-3] [INFO] [1785222288.848057264] [robot_state_publisher]: got segment imu_frame
[robot_state_publisher-3] [INFO] [1785222288.848063747] [robot_state_publisher]: got segment laser_frame
[robot_state_publisher-3] [INFO] [1785222288.848070240] [robot_state_publisher]: got segment lfwheel_Link
[robot_state_publisher-3] [INFO] [1785222288.848076888] [robot_state_publisher]: got segment rfwheel_Link
[joint_state_publisher-4] [INFO] [1785222289.372349515] [joint_state_publisher]: Waiting for robot_description to be published on the robot_description
[camera_driver-2] [INFO] [1785222289.716161193] [esp32_ip_camera_publisher]: camera node started, camera_url: http://192.168.12.37:81/stream
[camera_driver-2] [WARN] [1785222292.705741414] [esp32_ip_camera_publisher]: Camera not opened, trying reconnect... (next retry in 1.0s)
[camera_driver-2] [INFO] [1785222294.072701944] [esp32_ip_camera_publisher]: IP camera stream opened successfully
```

3. Start Gesture Action Group Control (Terminal 2)

```
ros2 launch microros_v2_no_ptz_visual_control_pkg
gesture_action_group_control.launch.py
```

4. Operation Instructions

Interaction

Key	Function
Space	waiting ↔ idle
i / I	Return to idle
q / Q / ESC	Quit (stop first)

Safety and Boundary Handling

- 5-finger emergency stop interrupts any action
- Gesture must be confirmed for ≥ 3 frames to trigger
- Odometry drives phase transitions
- Angle wraps $\pm\pi$
- Stop before quitting

Experiment Result

- **idle/waiting**: the car is stationary waiting for a gesture
- **Yes gesture held for 3 frames**: square 0.5m $\times$ 4 sides, looped once
- **OK gesture**: rotate 360° in place
- **Rock gesture**: S-curve
- **Thumb Down**: back-and-forth 0.5m
- **5-finger emergency stop**: immediately interrupts and returns to waiting

Code Explanation

Source paths:

- Launch:
`~/microros_ws/src/microros_v2_no_ptz_visual_control_pkg/launch/gesture_action_group_control.launch.py`
- Node source:
`~/microros_ws/src/microros_v2_no_ptz_visual_control_pkg/microros_v2_no_ptz_visual_control_pkg/gesture_action_group_control_node.py`
- Hand detector:
`~/microros_ws/src/microros_v2_no_ptz_visual_control_pkg/microros_v2_no_ptz_visual_control_pkg/hand_detector.py`

1. Core Node Analysis

`gesture_action_group_control_node` is the most complex node among the 4 documents, containing a **three-state machine** (idle/waiting/action_running), **dual timers** (image processing 15Hz + action control 30Hz), and an **odometry PD closed loop**.

Data flow:

- `_process_image` (15Hz) $\rightarrow$ `_detect` $\rightarrow$ MediaPipe gesture recognition $\rightarrow$ confirmation count $\rightarrow$ `_start_action`
- `_action_tick` (30Hz) $\rightarrow$ dispatches to `_square_tick/_circle_tick/_s_curve_tick/_backforth_tick` by action type $\rightarrow$ PD computation $\rightarrow$ `/cmd_vel`
- `_odom_callback` $\rightarrow$ caches odometry position and yaw

Constructor — declares parameters, creates dual timers, subscribers/publishers:

```
# Source file:
~/microros_ws/src/microros_v2_no_ptz_visual_control_pkg/microros_v2_no_ptz_visual_control_pkg/gesture_action_group_control_node.py

def __init__(self) -> None:
    super().__init__(DEFAULT_NODE_NAME)
    self._image_lock = threading.Lock()
```

```

self._state_lock = threading.Lock()
self._odom_lock = threading.Lock()

self._state = STATE_IDLE          # State machine initial: idle
self._latest_frame: Optional[np.ndarray] = None

# Odometry state (drives PD)
self._odom_x = 0.0
self._odom_y = 0.0
self._odom_yaw = 0.0

# Gesture confirmation mechanism
self._gesture_confirm_count = 0
self._last_gesture = ""

# Action engine state
self._action_type = ""
self._action_phase = 0
self._phase_start_x = 0.0
self._phase_start_y = 0.0
self._phase_start_yaw = 0.0
self._settle_count = 0

self._declare_params()
self._load_params()

self._hand_det = HandDetector()

# Subscriber QoS uniformly BEST_EFFORT (image + odometry)
best_effort_qos = QoSProfile(
    history=QoSHistoryPolicy.KEEP_LAST, depth=1,
    reliability=QoSReliabilityPolicy.BEST_EFFORT,
    durability=QoSDurabilityPolicy.VOLATILE,
)
self._img_sub = self.create_subscription(
    Image, self._img_topic, self._on_image, best_effort_qos)
self._odom_sub = self.create_subscription(
    Odometry, self._odom_topic, self._odom_callback, best_effort_qos)
self._cmd_pub = self.create_publisher(Twist, self._cmd_topic,
best_effort_qos)

# Dual timers: image processing 15Hz + action control 30Hz
self._img_timer = self.create_timer(
    1.0 / max(float(self._img_rate), MINIMUM_TIMER_RATE_HZ),
self._process_image)
self._action_timer = self.create_timer(
    1.0 / max(float(self._action_rate), MINIMUM_TIMER_RATE_HZ),
self._action_tick)

# Keyboard polling timer (50Hz)
if self._show_win:
    self._ensure_win()
    self._key_timer = self.create_timer(0.05, self._poll_keys)

self._param_cb = self.add_on_set_parameters_callback(self._on_param_update)

```

Action control timer callback — three-state machine scheduling:

```

# Source file:
~/microros_ws/src/microros_v2_no_ptz_visual_control_pkg/microros_v2_no_ptz_visua
l_control_pkg/gesture_action_group_control_node.py

def _action_tick(self) -> None:
    """Action control timer (30Hz), three-state machine scheduling."""
    if self._state == STATE_IDLE:
        self._pub_zero_cmd()      # idle: zero velocity
        return
    if self._state == STATE_WAITING:
        self._pub_zero_cmd()      # waiting: zero velocity, wait for gesture
        return
    if self._state != STATE_ACTION:
        return

    with self._odom_lock:
        ox = self._odom_x
        oy = self._odom_y
        oyaw = self._odom_yaw

    phase_dist = math.hypot(ox - self._phase_start_x, oy - self._phase_start_y)

    # Dispatch to different tick functions by action type
    if self._action_type == GESTURE_SQUARE:
        self._square_tick(oyaw, phase_dist)
    elif self._action_type == GESTURE_CIRCLE:
        self._circle_tick(oyaw)
    elif self._action_type == GESTURE_S:
        self._s_curve_tick(oyaw)
    elif self._action_type == GESTURE_BACKFORTH:
        self._backforth_tick(phase_dist)

```

Key logic: `_distance_pd` and `_angle_pd` compute the PD outputs for distance and angle respectively, with settle counting for determination. The S-curve action is an alternating turn→straight→turn→... sequence, driven by the `s_turn_angles` and `s_straight_distances` parameters.

2. Key Parameters

Parameter definition files:

- Launch:
`~/microros_ws/src/microros_v2_no_ptz_visual_control_pkg/launch/gesture_act
ion_group_control.launch.py`
- Node:
`~/microros_ws/src/microros_v2_no_ptz_visual_control_pkg/microros_v2_no_ptz
_visual_control_pkg/gesture_action_group_control_node.py`

Parameter	Type	Default	Description
linear_kp / linear_kd	float	0.8 / 0.25	Distance PD gains
angular_kp / angular_kd	float	0.05 / 0.005	Angle PD gains
max_linear_x	float	0.2 m/s	Maximum linear velocity
max_angular_z	float	0.6 rad/s	Maximum angular velocity
square_forward_distance	float	0.5 m	Square side length
circle_total_angle_deg	float	360.0°	Total rotation angle for the circle
backforth_distance	float	0.5 m	Back-and-forth distance
gesture_hold_confirm_frames	int	3	Gesture confirmation frame count
action_control_rate_hz	float	30.0 Hz	Action control rate
image_processing_rate_hz	float	15.0 Hz	Image processing rate
distance_tolerance	float	0.05 m	Distance determination tolerance
angle_tolerance_deg	float	3.0°	Angle determination tolerance
settle_frames	int	3	PD stabilization determination frames
phase_cooldown_ticks	int	10	Cooldown ticks between phases
s_turn_angles	list	[30,-60,60,-60,30]°	S-curve turn angle sequence
s_straight_distances	list	[0.288,0.576,0.576,0.288,0.5] m	S-curve straight distance sequence

3. Node Description

Node	Package	Description
gesture_action_group_control_node	microros_v2_no_ptz_visual_control_pkg	Gesture → action group, three-state machine + dual timers + PD closed loop

Common Problems

Problem	Cause	Solution
Action group interrupted	No data from <code>/odom_raw</code> odometry	Check whether the odometry topic is publishing normally
Gesture not triggered	Not enough confirmation frames	Hold the gesture for more than 3 frames
No detection at startup	MediaPipe environment not sourced	<code>`source /opt/ros/humble/setup.bash</code>
<code>source ~/microros_ws/install/setup.bash`</code>		